

Project Report 1

Shankaradithyaa Venkateswaran

October 2026

1 Pipeline Layout

We present a video detection and tracking pipeline in the domain of cell detection and tracking. The problem of cell detection and tracking has been explored before, but corresponding field of detecting and tracking cell division(mitosis) events has not received the same attention.

This project is an extension of the Kaggle BioHub 2026 competition that my partner and I took part of. The problem statement is to detect and track cell division events in a sparsely labeled dataset of 3 dimensional data through different time steps.

Dataset

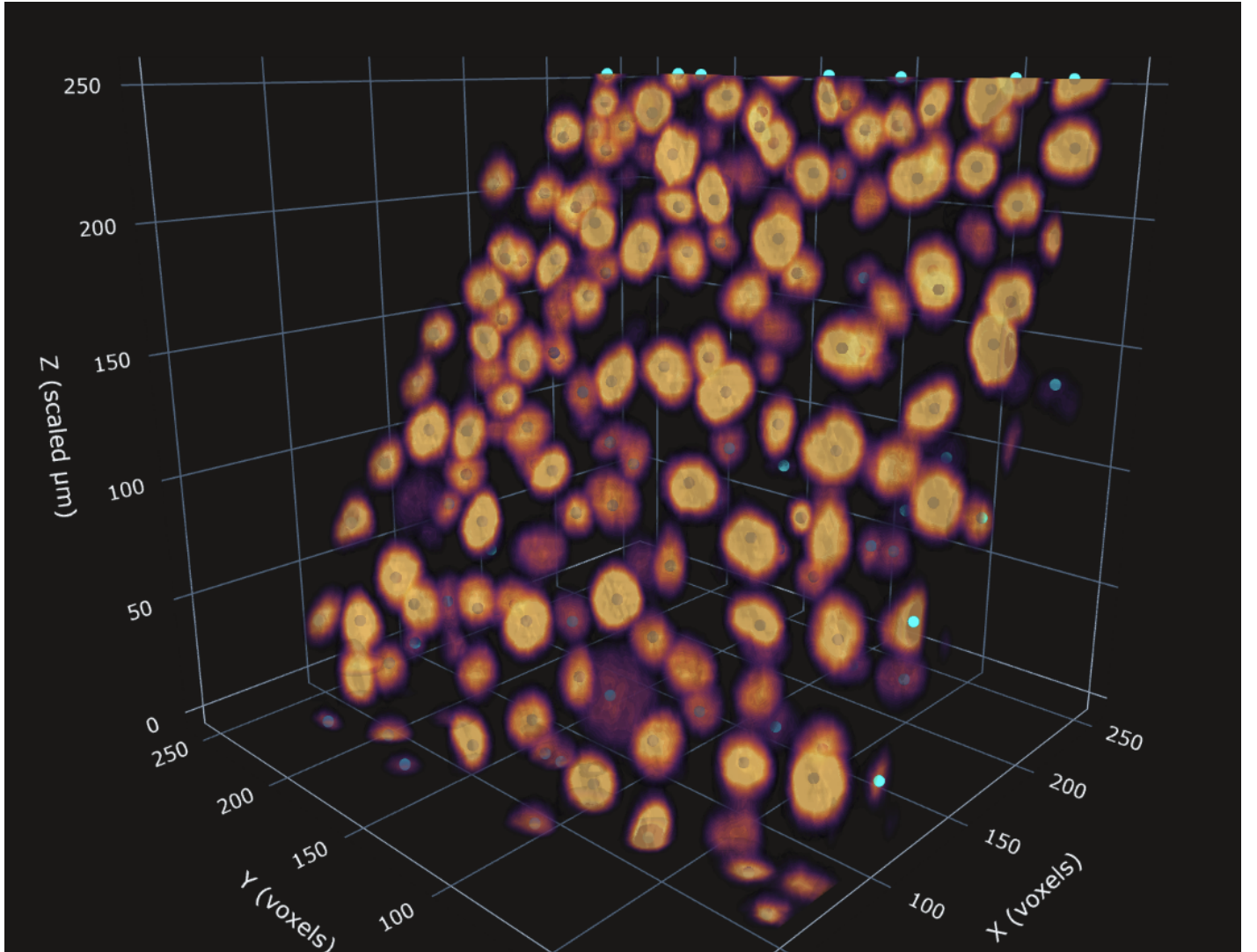


Figure 1: Dataset View

Image 1 shows a small cross section of the data, it is the output of a preliminary detection made through opencv on the data. A Difference of Gaussian(DoG) was applied on the dataset to get the colored representation of the cells that you see. The blue dots are the predicted portion of the cells that basic opencv method detects.

The dataset is called Biohub2026 and it is a collection of raw voxel data captured through florescence spectrography on the moving cells. The samples are zebra fish embryo that are growing while under monitoring, thus the cell divisions that occur. The dataset directory contains multiple embryos which are labeled as samples. Each embryo has multiple different field of views(fov) which they are viewed on that constitute as different data entries for the same embryo. A data entry for an embryo is its fov and the raw voxel data captured from that fov. All data entries are stored in a zarr data storage format which store the data across time steps, z, y, and x axis.

The scales are: time in seconds, and the physical scale is $z=1.625$, $y=0.40625$, $x=0.40625 \mu\text{m}/\text{voxel}$.

The ground truth annotations are stored similarly in a data format called geff. Geff is an extension of zarr that helps format the data stored in a more detailed manner. The main problem that we face here is that the number of ground truth annotations are truly sparse as compared to the size of the whole dataset. The number of estimated nodes are around 30,000 per data entry, while we only have a few hundred ground truth annotation to work with.

A note, we have access to some more open source data that is under our consideration to whether we should utilize it or not.

Models

We have two tasks that we can split this problem into. While they can be viewed as independent to each other, we think it is better to view both tasks as inter-related to each other.

Object Detection

The first task here is to detect a cell, there are plenty of methods, a sample of which you see at 1 but we need a method that will not over estimate the total number of cells and one that is accurate enough detect the cell centroids for further processing.

We do this by a combination of processes:

1. **DoG:** We can apply a Difference of Gaussians to the raw voxel data. We use this to generate a 3D map of the cells with which are better distinguishable from each other. Blobs of cells, where a lot of cells look like they are merged into each other are still a problem, but attacking that problem is currently outside the scope of this project.
2. **Intensity labeling:** Using the DoG data we can capture the intensity of a cell, that is during fluorescence spectrography, how much did the cell truly shine? The idea behind this is to capture the cell centers more accurately. We capture this as a feature to help any downstream models learn through the process better.
3. **Area of Overlap/Cell Movement Vectors:** This process is more of a batching processes rather than a live streaming application. To capitalize on this, we calculate something called the Area of Overlap which directly feed into capturing the cell motion vectors. A brief summary of this is to take a connected components segmentation of the 3D cell map at each time step. We map each connected component to its match across consecutive time steps using minimal distance, maximal pairing bipartite matching. Using these matched connected components of each pair across consecutive time steps we calculate the area that they overlap by and the calculate the motion vectors of each connected component. This information gives the model information about the future from the current time step it is calculating on.

A base line method for detection is just using this features to predict cell centroids using a suitable OpenCV method.

With these input features calculated from the data we feed it into an object detection model. The main architecture that we chose is the 3D Unet Model [1]. The architecture is known to work well on 3D volumetric data which have sparse data annotations.

A point to note is that this model architecture predicts a 3D volumetric segmentation of the input data. We will run a classifier layer upon this 3D layer to predict the actual locations of cells during and after cell division. This entire process is to specifically predict the cells that undergo divisions in a specified window of before and after the division event.

To further improve the model, we can run this 3D Unet in the form of an LSTM unit to pass the past information to the future while also allowing the model to maintain a one step look ahead provided through the features we provide.

We are also in the process of exploring ImageJ2/Fiji which is a software that is widely used to perform cell detection and tracking and we are looking forward to integrate it into our pipelines.

Object Tracking

The next step of the process after cell detection is to accurately track the cells through the 3D space across the different time steps.

The tricky part here is that we must maintain cell tracking with several rules. The method that we follow is to predict the cells accurately (from the object detector) and then perform a cell tracking operation on them for now.

We use a Kalman Filter [2] for tracking. The Filter estimates the state of an individual track and then predicts its next position. Using this prediction is associated with the incoming annotation passed by the object detector to reduce total error. Overall, the Filter helps manage the tracks that we are detecting.

We are also exploring the option to develop a deep learning tracker that can learn some pattern for cell tracking, but that is for a future work.

There are several heuristic rules the tracking must follow, which we will discuss in the next section.

Assertions

The tracking produced at the end cannot violate some set rules, we can use these rules as assertions to get:

1. The total number of nodes predicted cannot exceed a threshold factor of 10 percent of the total estimated nodes.
2. Cells cannot merge into one another. Thus, the produced lineage graph via the Object Tracker cannot have nodes merge into one another. This is another way to say that a cell cannot have two parents.
3. Each tracked parent cell must have two immediate daughters (one node breaks into two) and the daughters must each have a singular immediate grandchild. Another way to say this is that a cell division cannot take place in consecutive time steps for biological reasons.
4. The tracked parent must be in a time step before the children are tracked. Merely having a weakly connected component is not good enough.
5. The tracked daughter cells must be within a reasonable distance from their parent. I.e. The daughters can't fly off into a completely different part of the embryo within a couple of time steps.

Configurations

The total number of pipeline configurations we have so far are as follows:

Dataset	1
Feature Engineering	3
Object Detection	2
Object Tracking	1
Total	6

Table 1: Configurations

I have not taken into consideration the models and choices that are still under thought, but they will be added later on if needed.

2 Accuracy

The pipeline has not been run with the current assertions as the accuracy metric. Since we took part in the competition, we were focusing on getting evaluated with the ground truth. Our final score with just the base OpenCV Detector and the Kalman Tracker was 0.769 on the test set.

This section will be populated once we have the results.

3 Compute Platform and SLAs

The current pipelines were run on Kaggle with 2 NVIDIA T4 gpus as support. This is expected to change as the platform is shifted to Chameleon or Hellbender.

4 Github

The notebook labeled **Submission_Notebook.ipynb** can be found at my Github: OmegaSun18/BioHub2026-Competition.

5 Teamwork Organization

The main form of communication I had with my partner was Discord along with a google document to organize our ideas and results. A huge thanks for my partner Himanish Goel for participating in the competition with me. He focused on creating the base features of DoG and Intensity along with the base OpenCV detection and the Kalman Tracker. I focused on literature review, the Area of Intersection feature, and the transformer based detection models.

References

- [1] Özgün Cicek, Ahmed Abdulkadir, Soeren S. Lienkamp, Thomas Brox, and Olaf Ronneberger. 3D U-Net: Learning Dense Volumetric Segmentation from Sparse Annotation, 2016.
- [2] Rudolph E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960.